Sphercity_testing

Harshika

2026-04-14

R Markdown

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(purrr)

base <- getwd()
print(base)
```

```
## [1] "/home/samaharshika/Documents/BRSM/Project"
```

```
path <- file.path(base, "data_brsm", "single", "phone")

files <- sort(list.files(path = path,
                         pattern = "\\.csv$",
                         full.names = TRUE))

# print(files)


path1 <- file.path(getwd(), "data_brsm", "multiple", "phone")

multiple_files <- sort(list.files(path = path1,
                         pattern = "\\.csv$",
                         full.names = TRUE))

# print(multiple_files)
```

```r
single_phone <- map_df(files, function(f) {
  read.csv(f, stringsAsFactors = FALSE)
}, .id = "Player")


single_phone <- single_phone %>%
  select(Player, PlayerID, Timestamp, GameMode, Level, FinalScore,
         Completed, SuccessRate... , HitRate..., FalseAlarms,
         InitialResponseTime.ms., AvgInterTargetTime.ms., HitPositions.x.y.)

# str(single_phone)
length(unique(single_phone$Player))
```

```
## [1] 21
```

```r
table(single_phone$GameMode)
```

```
##
## singleTarget
##          312
```

```r
table(single_phone$Player)
```

```
##
##  1 10 11 12 13 14 15 16 17 18 19  2 20 21  3  4  5  6  7  8  9
## 15 16 15 16 15 15 16 16 19 19 15 15 18 15 15 15 15 15 16 15 15
```

```r
colnames(single_phone[7])
```

```
## [1] "Completed"
```

```r
names(single_phone) <- make.names(names(single_phone))
names(single_phone)
```

```
##  [1] "Player"                  "PlayerID"
##  [3] "Timestamp"               "GameMode"
##  [5] "Level"                   "FinalScore"
##  [7] "Completed"               "SuccessRate..."
##  [9] "HitRate..."              "FalseAlarms"
## [11] "InitialResponseTime.ms." "AvgInterTargetTime.ms."
## [13] "HitPositions.x.y."
```

```r
 # print(single_phone)
library(dplyr)

table(single_phone$GameMode)
```

```
##
## singleTarget
##          312
```

```r
names(single_phone) <- make.names(names(single_phone))
#names(single_phone)

single_phone$SuccessRate... <- as.numeric(single_phone$SuccessRate...)
single_phone$HitRate... <- as.numeric(single_phone$HitRate...)

single_phone$SuccessRate... <- as.numeric(single_phone$SuccessRate...)
single_phone$HitRate... <- as.numeric(single_phone$HitRate...)
```

Including Plots

```r
library(ez)
```

```
## Registered S3 method overwritten by 'lme4':
##   method           from
##   na.action.merMod car
```

```r
library(dplyr)
library(purrr)



# length(unique(single_phone_new$PlayerID))
names(single_phone)
```

```
##  [1] "Player"                  "PlayerID"
##  [3] "Timestamp"               "GameMode"
##  [5] "Level"                   "FinalScore"
##  [7] "Completed"               "SuccessRate..."
##  [9] "HitRate..."              "FalseAlarms"
## [11] "InitialResponseTime.ms." "AvgInterTargetTime.ms."
## [13] "HitPositions.x.y."
```

```r
data_game <- single_phone %>%
  filter(
    GameMode == "singleTarget",
    Completed == "true"
  ) %>%
  group_by(Player, Level) %>%
  summarise(
    InitialResponseTime.ms. = mean(InitialResponseTime.ms., na.rm = TRUE)
  ) %>%
  ungroup()
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Player and Level.
## i Output is grouped by Player.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Player, Level))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```r
# data_game <- single_phone_new %>%
#  filter(GameMode == "singleTarget") %>%
 # select(PlayerID, Level, InitialResponseTime.ms.) %>%
#  na.omit()

data_game$Player <- as.factor(data_game$Player)
data_game$Level <- as.factor(data_game$Level)

library(ez)

anova_result <- ezANOVA(
  data = data_game,
  dv = InitialResponseTime.ms.,
  wid = Player,
  within = .(Level), # <--- This triggers the test
  detailed = TRUE
)

# Formula: Dependent Variable ~ Within-Subject Factor | Subject ID
friedman.test(InitialResponseTime.ms. ~ Level | Player, data = data_game)
```

```
##
##  Friedman rank sum test
##
## data:  InitialResponseTime.ms. and Level and Player
## Friedman chi-squared = 154.77, df = 14, p-value < 2.2e-16
```

```r
# Run pairwise comparisons with a Holm correction to adjust for multiple tests
pairwise.wilcox.test(
  x = data_game$InitialResponseTime.ms.,
  g = data_game$Level,
  p.adjust.method = "holm",
  paired = TRUE
)
```

```
## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with ties
## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with ties
## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with ties
## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with ties
## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with ties
## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with ties

##
##  Pairwise comparisons using Wilcoxon signed rank exact test
##
## data:  data_game$InitialResponseTime.ms. and data_game$Level
```

```
##
##    1       2       3       4       5       6       7       8       9
## 2  0.00591 -       -       -       -       -       -       -       -
## 3  1.00000 0.00020 -       -       -       -       -       -       -
## 4  0.00051 0.00020 0.03750 -       -       -       -       -       -
## 5  1.00000 0.00020 0.54466 0.01672 -       -       -       -       -
## 6  0.98575 0.00036 1.00000 0.14188 1.00000 -       -       -       -
## 7  0.00372 0.00020 0.06046 1.00000 0.02408 0.07943 -       -       -
## 8  1.00000 0.00036 1.00000 0.00036 1.00000 1.00000 0.02751 -       -
## 9  1.00000 0.01041 1.00000 0.02751 1.00000 1.00000 0.06876 1.00000 -
## 10 0.00162 0.00020 0.00083 0.54466 0.00020 0.00290 1.00000 0.00162 0.00591
## 11 0.06046 0.00020 0.00935 1.00000 0.15714 1.00000 1.00000 0.02751 0.30642
## 12 0.04338 0.00020 0.01141 1.00000 0.01672 0.07943 1.00000 0.01530 0.00372
## 13 0.00472 0.00020 0.01672 1.00000 0.02426 0.02408 1.00000 0.00472 0.00051
## 14 0.02408 0.00020 0.00162 1.00000 0.01141 0.01385 1.00000 0.00372 0.00290
## 15 0.00216 0.00020 0.00083 0.02751 0.00162 0.00472 0.14188 0.00036 0.00051
##    10      11      12      13      14
## 2  -       -       -       -       -
## 3  -       -       -       -       -
## 4  -       -       -       -       -
## 5  -       -       -       -       -
## 6  -       -       -       -       -
## 7  -       -       -       -       -
## 8  -       -       -       -       -
## 9  -       -       -       -       -
## 10 -       -       -       -       -
## 11 1.00000 -       -       -       -
## 12 1.00000 1.00000 -       -       -
## 13 1.00000 1.00000 1.00000 -       -
## 14 1.00000 1.00000 1.00000 1.00000 -
## 15 1.00000 0.30642 0.03750 1.00000 0.54466
##
## P value adjustment method: holm
```

```
print(anova_result)
```

```
## $ANOVA
##        Effect DFn DFd        SSn       SSd         F            p p<.05
## 1 (Intercept)   1  19 2821848557 108841469 492.59830 4.760011e-15     *
## 2       Level  14 266  643213196 968016340  12.62484 1.448182e-22     *
##         ges
## 1 0.7237910
## 2 0.3739457
##
## $`Mauchly's Test for Sphericity`
##   Effect            W           p p<.05
## 2  Level 1.033389e-09 8.562747e-19     *
##
## $`Sphericity Corrections`
##   Effect       GGe        p[GG] p[GG]<.05       HFe        p[HF] p[HF]<.05
## 2  Level 0.3626448 1.606407e-09         * 0.5106695 1.435583e-12         *
```

```r
multiple_phone <- map_df(multiple_files, function(f) {
  read.csv(f, stringsAsFactors = FALSE)
}, .id = "Player")

# 2. Clean the column names so they perfectly match single_phone
names(multiple_phone) <- make.names(names(multiple_phone))

# 3. Format the data for the Friedman test (Now 'Player' exists!)
data_game_multi <- multiple_phone %>%
  filter(tolower(as.character(Completed)) == "true") %>% # Safety check for True/true
  group_by(Player, Level) %>%
  summarise(
    InitialResponseTime.ms. = mean(InitialResponseTime.ms., na.rm = TRUE)
  ) %>%
  ungroup()
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Player and Level.
## i Output is grouped by Player.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Player, Level))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```r
library(dplyr)
library(purrr)

# 1. Load the CSVs and generate the "Player" column
multiple_phone <- map_df(multiple_files, function(f) {
  read.csv(f, stringsAsFactors = FALSE)
}, .id = "Player")

# 2. Clean the column names
names(multiple_phone) <- make.names(names(multiple_phone))

# 3. CRITICAL STEP: Drop any level that was not successfully completed
data_game_multi <- multiple_phone %>%
  filter(tolower(as.character(Completed)) == "true") %>% # <--- THIS DROPS INCOMPLETE LEVELS
  group_by(Player, Level) %>%
  summarise(
    InitialResponseTime.ms. = mean(InitialResponseTime.ms., na.rm = TRUE)
  ) %>%
  ungroup()
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Player and Level.
## i Output is grouped by Player.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Player, Level))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```r
# 4. Cap the analysis to Levels 1 through 10
capped_multi <- data_game_multi %>%
```

```
  filter(as.numeric(as.character(Level)) <= 10)

# 5. Find ONLY the players who successfully completed all 10 capped levels
complete_players <- capped_multi %>%
  group_by(Player) %>%
  tally() %>%
  filter(n == 10) %>% # They must have survived exactly 10 levels
  pull(Player)

# 6. Filter the dataset to keep only those "survivor" players
balanced_multi <- capped_multi %>%
  filter(Player %in% complete_players)

# 7. Convert to factors and drop the "ghost" data
balanced_multi$Player <- droplevels(as.factor(balanced_multi$Player))
balanced_multi$Level <- droplevels(as.factor(balanced_multi$Level))

# 8. Run the Friedman Test on the perfectly balanced, fully completed data!
cat("\n--- FRIEDMAN TEST: MULTIPLE TARGET GAME (LEVELS 1-10) ---\n")
```

```
##
## --- FRIEDMAN TEST: MULTIPLE TARGET GAME (LEVELS 1-10) ---
```

```
print(friedman.test(InitialResponseTime.ms. ~ Level | Player, data = balanced_multi))
```

```
##
##  Friedman rank sum test
##
## data:  InitialResponseTime.ms. and Level and Player
## Friedman chi-squared = 29.896, df = 9, p-value = 0.000457
```

```
# Post hoc pairwise comparisons
pairwise.wilcox.test(
  x = balanced_multi$InitialResponseTime.ms.,
  g = balanced_multi$Level,
  paired = TRUE,
  p.adjust.method = "holm"
)
```

```
## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with ties
```

```
##
##  Pairwise comparisons using Wilcoxon signed rank exact test
##
## data:  balanced_multi$InitialResponseTime.ms. and balanced_multi$Level
##
##    1     2     3     4     5     6     7     8     9
## 2  0.280 -     -     -     -     -     -     -     -
## 3  1.000 0.082 -     -     -     -     -     -     -
## 4  1.000 0.280 1.000 -     -     -     -     -     -
## 5  1.000 0.044 1.000 1.000 -     -     -     -     -
```

```
## 6  1.000 0.280 1.000 1.000 1.000 -     -     -     -
## 7  1.000 1.000 1.000 1.000 0.520 1.000 -     -     -
## 8  1.000 0.044 1.000 1.000 1.000 1.000 1.000 -     -
## 9  1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 -
## 10 1.000 0.044 1.000 1.000 1.000 1.000 1.000 1.000 1.000
##
## P value adjustment method: holm
```

```r
# Assuming you have loaded 'single_phone' and 'multiple_phone' dataframes
# First, create a new column in each to identify the group
single_phone$TargetLoad <- "Single"
multiple_phone$TargetLoad <- "Multiple"

# Bind them together into one large dataframe
# combined_data <- rbind(single_phone, multiple_phone)

combined_data <- bind_rows(single_phone, multiple_phone)

# Make sure it's a factor
combined_data$TargetLoad <- as.factor(combined_data$TargetLoad)

# Ensure the grouping variable is a factor
combined_data$TargetLoad <- as.factor(combined_data$TargetLoad)

# Get a list of all unique levels in your game
levels_list <- sort(unique(combined_data$Level))

# Loop through each level and run the test
# Get a list of all unique levels in your game
levels_list <- sort(unique(combined_data$Level))

levels_list <- 1:10
# Loop through each level and run the test
for (lvl in levels_list) {

  # Filter data for the current level
  lvl_data <- combined_data %>% filter(Level == lvl)

  # Run the test (paired = FALSE is removed)
  test_result <- wilcox.test(
    InitialResponseTime.ms. ~ TargetLoad,
    data = lvl_data,
    exact = FALSE
  )

  # Check if it is statistically significant
  significance <- ifelse(test_result$p.value < 0.05, "* Significant", "Not Significant")

  # Print the result for this level
  cat("Level:", lvl,
      "| W =", round(test_result$statistic, 2),
      "| p-value =", formatC(test_result$p.value, format = "e", digits = 2),
      "|", significance, "\n")
}
```

```
## Level: 1 | W = 162.5 | p-value = 9.49e-01 | Not Significant
## Level: 2 | W = 163 | p-value = 9.37e-01 | Not Significant
## Level: 3 | W = 163 | p-value = 9.37e-01 | Not Significant
## Level: 4 | W = 114 | p-value = 4.00e-02 | * Significant
## Level: 5 | W = 150 | p-value = 5.52e-01 | Not Significant
## Level: 6 | W = 101.5 | p-value = 3.82e-02 | * Significant
## Level: 7 | W = 52 | p-value = 2.16e-04 | * Significant
## Level: 8 | W = 104 | p-value = 7.72e-02 | Not Significant
## Level: 9 | W = 69 | p-value = 3.96e-03 | * Significant
## Level: 10 | W = 105 | p-value = 9.17e-05 | * Significant
```

```
base <- getwd()
path <- file.path(base, "data_brsm", "single", "lab")
files <- sort(list.files(path = path,
                         pattern = "\\.csv$",
                         full.names = TRUE))

single_lab_rt <- c()
single_lab_acc <- c()
single_lab_fr <- c()
single_levelwise_rt <- NULL

for (file in files) {

  df <- read.csv(file)
  entries <- df$mouse.clicked_name
  entries <- entries[!is.na(entries)]

  all_clicks <- c()
  for (entry in entries) {

    clean <- gsub("\\[|\\]|'", "", entry)
    clicks <- strsplit(clean, ",")[[1]]
    clicks <- trimws(clicks)

    all_clicks <- c(all_clicks, clicks)
  }

  total_clicks <- length(all_clicks)
  hits <- sum(all_clicks == "target")
  false_clicks <- total_clicks - hits

  accuracy <- hits / total_clicks
  false_alarm_rate <- false_clicks / total_clicks

  single_lab_acc <- c(single_lab_acc, accuracy)
  single_lab_fr  <- c(single_lab_fr, false_alarm_rate)
  response_times <- df$mouse.time
  response_times <- response_times[!is.na(response_times)]
  response_times <- gsub("\\[|\\]", "", response_times)
  response_times <- as.numeric(response_times)

  single_levelwise_rt <- rbind(single_levelwise_rt, response_times)
  single_lab_rt <- c(single_lab_rt,
```

```
                          mean(response_times, na.rm = TRUE))
}

print(mean(single_lab_rt))
```

```
## [1] 1.56541
```

```
path1 <- file.path(getwd(), "data_brsm", "multiple", "lab")

files <- sort(list.files(path = path1,
                         pattern = "\\.csv$",
                         full.names = TRUE))

multi_lab_rt <- c()
multi_lab_acc <- c()
multi_levelwise_rt <- NULL

for (file in files) {

  df <- read.csv(file, stringsAsFactors = FALSE)

  # ----- RT calculation -----

  all_values <- c()
  entries <- df$mouse.time
  entries <- entries[!is.na(entries)]
  for (entry in entries) {

    split_lists <- strsplit(gsub("\\]\\[", "]|||[", entry), "\\|\\|\\|")[[1]]

    for (lst in split_lists) {

      clean <- gsub("\\[|\\]", "", lst)

      if (nchar(clean) > 0) {
        parsed <- as.numeric(strsplit(clean, ",")[[1]])

        if (length(parsed) > 0) {
          s <- tail(parsed, 1)
          all_values <- c(all_values, s / length(parsed))
        }
      }
    }
  }
  multi_levelwise_rt <- rbind(multi_levelwise_rt, all_values)
  multi_lab_rt <- c(multi_lab_rt, mean(all_values))


  entries1 <- df$mouse.clicked_name
  entries1 <- entries1[!is.na(entries1)]

  round_acc <- c()
```

```
  for (entry in entries1) {

    clean <- gsub("\\[|\\]|'", "", entry)
    clicks <- strsplit(clean, ",")[[1]]
    clicks <- trimws(clicks)

    unique_targets <- length(unique(clicks))

    acc <- unique_targets / 5

    round_acc <- c(round_acc, acc)
  }

  multi_lab_acc <- c(multi_lab_acc, mean(round_acc))
}
```

```
library(tidyr)
library(dplyr)

# Set how many levels we expect a "complete" player to have survived
target_levels <- 9

# ========================================================
# 2A. CRASH-PROOF FRIEDMAN: SINGLE TARGET LAB (Capped at 10)
# ========================================================
# Convert matrix to dataframe and add Player ID
single_lab_df <- as.data.frame(single_levelwise_rt)
single_lab_df$Player <- as.factor(1:nrow(single_lab_df))

# Explicitly set the cap to 10 to match your Game analysis
target_levels_single <- 9

# Pivot to long format and clean
single_lab_long <- pivot_longer(
  single_lab_df,
  cols = -Player,
  names_to = "Level_Raw",
  values_to = "RT"
) %>%
  drop_na(RT) %>%
  mutate(Level = as.numeric(gsub("V", "", Level_Raw))) %>%
  filter(Level <= target_levels_single) # <--- Caps it at Level 10

# Find ONLY the players who completed all 10 levels
complete_single_lab_players <- single_lab_long %>%
  group_by(Player) %>%
  tally() %>%
  filter(n == target_levels_single) %>%
  pull(Player)

# Filter out the quitters, drop ghosts, and run the test
balanced_single_lab <- single_lab_long %>%
  filter(Player %in% complete_single_lab_players) %>%
```

```
  mutate(Player = droplevels(as.factor(Player)),
         Level = droplevels(as.factor(Level)))

cat("\n--- FRIEDMAN TEST: SINGLE TARGET LAB (LEVELS 1-10) ---\n")
```

```
##
## --- FRIEDMAN TEST: SINGLE TARGET LAB (LEVELS 1-10) ---
```

```
if(nrow(balanced_single_lab) > 0) {
    print(friedman.test(RT ~ Level | Player, data = balanced_single_lab))

    pairwise.wilcox.test(
      x = balanced_single_lab$InitialResponseTime.ms.,
      g = balanced_single_lab$Level,
      paired = TRUE,
      p.adjust.method = "holm"
    )
} else {
    cat("Error: We need to lower the cap. No players completed", target_levels_single, "levels.\n")
}
```

```
## Error: We need to lower the cap. No players completed 9 levels.
```

```
# ========================================================
# 2B. CRASH-PROOF FRIEDMAN: MULTIPLE TARGET LAB
# ========================================================
# Convert matrix to dataframe and add Player ID
multi_lab_df <- as.data.frame(multi_levelwise_rt)
multi_lab_df$Player <- as.factor(1:nrow(multi_lab_df))

# Automatically detect how many levels the multi lab has
target_levels_multi <- ncol(multi_levelwise_rt)

# Pivot and clean
multi_lab_long <- pivot_longer(
  multi_lab_df,
  cols = -Player,
  names_to = "Level_Raw",
  values_to = "RT"
) %>%
  drop_na(RT) %>%
  mutate(Level = as.numeric(gsub("V", "", Level_Raw))) %>%
  filter(Level <= target_levels_multi)

# Find ONLY the players who completed all levels
complete_multi_lab_players <- multi_lab_long %>%
  group_by(Player) %>%
  tally() %>%
  filter(n == target_levels_multi) %>%
  pull(Player)

# Filter, drop ghosts, and run the test
```

```r
balanced_multi_lab <- multi_lab_long %>%
  filter(Player %in% complete_multi_lab_players) %>%
  mutate(Player = droplevels(as.factor(Player)),
         Level = droplevels(as.factor(Level)))

cat("\n--- FRIEDMAN TEST: MULTIPLE TARGET LAB ---\n")
```

```
##
## --- FRIEDMAN TEST: MULTIPLE TARGET LAB ---
```

```r
if(nrow(balanced_multi_lab) > 0) {
    print(friedman.test(RT ~ Level | Player, data = balanced_multi_lab))

    pairwise.wilcox.test(
      x = balanced_multi_lab$RT,
      g = balanced_multi_lab$Level,
      paired = TRUE,
      p.adjust.method = "holm"
    )

} else {
    cat("Error: No players successfully completed all", target_levels_multi, "levels.\n")
}
```

```
##
##  Friedman rank sum test
##
## data:  RT and Level and Player
## Friedman chi-squared = 37.375, df = 14, p-value = 0.0006472

##
##  Pairwise comparisons using Wilcoxon signed rank exact test
##
## data:  balanced_multi_lab$RT and balanced_multi_lab$Level
##
##    1     2     3     4     5     6     7     8     9     10    11    12
## 2  1.000 -     -     -     -     -     -     -     -     -     -     -
## 3  1.000 1.000 -     -     -     -     -     -     -     -     -     -
## 4  1.000 1.000 1.000 -     -     -     -     -     -     -     -     -
## 5  1.000 1.000 1.000 1.000 -     -     -     -     -     -     -     -
## 6  1.000 1.000 1.000 1.000 1.000 -     -     -     -     -     -     -
## 7  1.000 1.000 1.000 1.000 1.000 1.000 -     -     -     -     -     -
## 8  0.016 0.269 0.135 1.000 1.000 1.000 1.000 -     -     -     -     -
## 9  0.079 0.616 0.616 1.000 1.000 1.000 1.000 1.000 -     -     -     -
## 10 0.414 0.873 0.218 1.000 1.000 1.000 1.000 1.000 1.000 -     -     -
## 11 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 -     -
## 12 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 -
## 13 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000
## 14 1.000 1.000 0.616 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000
## 15 0.218 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000
##    13    14
## 2  -     -
```

```
## 3  -     -
## 4  -     -
## 5  -     -
## 6  -     -
## 7  -     -
## 8  -     -
## 9  -     -
## 10 -     -
## 11 -     -
## 12 -     -
## 13 -     -
## 14 1.000 -
## 15 1.000 1.000
##
## P value adjustment method: holm
```

```r
single_lab_long %>%
  group_by(Player) %>%
  summarise(Completed_Levels = sum(!is.na(RT))) %>%
  count(Completed_Levels, name = "Number_of_Players") %>%
  arrange(desc(Completed_Levels))
```

```
## # A tibble: 1 x 2
##   Completed_Levels Number_of_Players
##              <int>             <int>
## 1                8                21
```

```r
library(tidyr)
library(dplyr)

# 1. Convert matrix to dataframe and add Player ID
single_lab_df <- as.data.frame(single_levelwise_rt)
single_lab_df$Player <- as.factor(1:nrow(single_lab_df))

# 2. Pivot to long format and strip out NAs
single_lab_long <- pivot_longer(
  single_lab_df,
  cols = -Player,
  names_to = "Level_Raw",
  values_to = "RT"
) %>%
  drop_na(RT) %>%
  mutate(Level = as.numeric(gsub("V", "", Level_Raw)))

# 3. THE SMART FIX: Find exactly which levels have 0 missing data
# Count how many total players we have (should be 21 based on your image)
total_players <- length(unique(single_lab_long$Player))

# Find only the levels where the count perfectly matches the total players
complete_levels <- single_lab_long %>%
  group_by(Level) %>%
  tally() %>%
  filter(n == total_players) %>%
```

```
  pull(Level)

# 4. Filter the dataset to ONLY include those fully complete levels
balanced_single_lab <- single_lab_long %>%
  filter(Level %in% complete_levels) %>%
  mutate(
    Player = droplevels(as.factor(Player)),
    Level = droplevels(as.factor(Level))
  )

# 5. Run the test!
cat("\n--- FRIEDMAN TEST: SINGLE TARGET LAB (COMPLETE LEVELS ONLY) ---\n")
```

```
##
## --- FRIEDMAN TEST: SINGLE TARGET LAB (COMPLETE LEVELS ONLY) ---
```

```
if(length(complete_levels) > 1) {
    # Print out which levels it actually found so you can check PsychoPy's numbering
    cat("Successfully found balanced data for Levels:", paste(complete_levels, collapse=", "), "\n\n")
    print(friedman.test(RT ~ Level | Player, data = balanced_single_lab))
    pairwise.wilcox.test(
      x = balanced_single_lab$RT,
      g = balanced_single_lab$Level,
      paired = TRUE,
      p.adjust.method = "holm"
    )
} else {
    cat("Error: Could not find enough complete levels. Check raw data matrix.\n")
}
```

```
## Successfully found balanced data for Levels: 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16
##
##
##  Friedman rank sum test
##
## data:  RT and Level and Player
## Friedman chi-squared = 51.076, df = 14, p-value = 4.027e-06

##
##  Pairwise comparisons using Wilcoxon signed rank exact test
##
## data:  balanced_single_lab$RT and balanced_single_lab$Level
##
##    2      3      4      5      6      7      8      9      10     11     12
## 3  1.0000 -      -      -      -      -      -      -      -      -      -
## 4  0.0239 1.0000 -      -      -      -      -      -      -      -      -
## 5  0.0014 0.0019 1.0000 -      -      -      -      -      -      -      -
## 6  0.0792 1.0000 1.0000 1.0000 -      -      -      -      -      -      -
## 7  0.4342 1.0000 1.0000 1.0000 1.0000 -      -      -      -      -      -
## 8  0.0583 1.0000 1.0000 1.0000 1.0000 1.0000 -      -      -      -      -
## 9  0.0054 0.8910 1.0000 1.0000 1.0000 1.0000 1.0000 -      -      -      -
## 10 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 -      -      -
```

```
## 11 0.0014 0.1936 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 -      -
## 12 0.1710 0.8910 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 -
## 13 0.0287 0.1710 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000
## 14 0.0197 0.0685 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000
## 15 0.0054 0.0685 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000
## 16 0.0287 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000
##    13     14     15
## 3  -      -      -
## 4  -      -      -
## 5  -      -      -
## 6  -      -      -
## 7  -      -      -
## 8  -      -      -
## 9  -      -      -
## 10 -      -      -
## 11 -      -      -
## 12 -      -      -
## 13 -      -      -
## 14 1.0000 -      -
## 15 1.0000 1.0000 -
## 16 1.0000 1.0000 1.0000
##
## P value adjustment method: holm
```

```
library(dplyr)
library(tidyr)

cat("\n--- LAB TASK: SINGLE VS MULTIPLE (MATCHING LEVELS ONLY) ---\n")
```

```
##
## --- LAB TASK: SINGLE VS MULTIPLE (MATCHING LEVELS ONLY) ---
```

```
# 1. Find the levels that exist in BOTH the Single and Multi datasets
# (This prevents R from crashing if one group played a level the other didn't)
common_levels <- intersect(unique(single_lab_long$Level), unique(multi_lab_long$Level))
common_levels <- sort(common_levels)

# 2. Loop through only the overlapping levels
for (lvl in common_levels) {

  # Extract the reaction times for just this specific level
  single_lvl_data <- single_lab_long %>% filter(Level == lvl) %>% pull(RT)
  multi_lvl_data <- multi_lab_long %>% filter(Level == lvl) %>% pull(RT)

  # Ensure we have data in both groups to compare
  if (length(single_lvl_data) > 0 & length(multi_lvl_data) > 0) {

    # Run the independent Mann-Whitney test
    test_result <- wilcox.test(single_lvl_data, multi_lvl_data, exact = FALSE)

    # Check significance
    significance <- ifelse(test_result$p.value < 0.05, "* Significant", "Not Significant")
```

```
    # Print the clean output
    cat("Lab Level:", sprintf("%-2s", lvl),
        "| W =", sprintf("%-6s", round(test_result$statistic, 2)),
        "| p-value =", formatC(test_result$p.value, format = "e", digits = 2),
        "|", significance, "\n")

  } else {
    cat("Lab Level:", sprintf("%-2s", lvl), "| Skipped: Missing data in one group.\n")
  }
}
```

```
## Lab Level: 2  | W = 262    | p-value = 4.15e-03 | * Significant
## Lab Level: 3  | W = 227    | p-value = 7.29e-02 | Not Significant
## Lab Level: 4  | W = 235    | p-value = 4.15e-02 | * Significant
## Lab Level: 5  | W = 203    | p-value = 2.90e-01 | Not Significant
## Lab Level: 6  | W = 246    | p-value = 1.75e-02 | * Significant
## Lab Level: 7  | W = 206    | p-value = 2.50e-01 | Not Significant
## Lab Level: 8  | W = 273    | p-value = 1.36e-03 | * Significant
## Lab Level: 9  | W = 271    | p-value = 1.68e-03 | * Significant
## Lab Level: 10 | W = 243    | p-value = 2.24e-02 | * Significant
## Lab Level: 11 | W = 195    | p-value = 4.17e-01 | Not Significant
## Lab Level: 12 | W = 234    | p-value = 4.46e-02 | * Significant
## Lab Level: 13 | W = 219    | p-value = 1.22e-01 | Not Significant
## Lab Level: 14 | W = 237    | p-value = 3.57e-02 | * Significant
## Lab Level: 15 | W = 214    | p-value = 1.63e-01 | Not Significant
```

```
# ========================================
# 1. PHONE (GAME) DATA
# ========================================
# Single Target Phone
single_phone_levels <- sort(unique(as.numeric(as.character(single_phone$Level))))
cat("--- SINGLE PHONE (GAME) ---\n")
```

```
## --- SINGLE PHONE (GAME) ---
```

```
cat("Total Number of Levels:", length(single_phone_levels), "\n")
```

```
## Total Number of Levels: 15
```

```
cat("Level Numbers:", single_phone_levels, "\n\n")
```

```
## Level Numbers: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
```

```
# Multiple Target Phone
multi_phone_levels <- sort(unique(as.numeric(as.character(multiple_phone$Level))))
cat("--- MULTIPLE PHONE (GAME) ---\n")
```

```
## --- MULTIPLE PHONE (GAME) ---
```

```r
cat("Total Number of Levels:", length(multi_phone_levels), "\n")
```

```
## Total Number of Levels: 14
```

```r
cat("Level Numbers:", multi_phone_levels, "\n\n")
```

```
## Level Numbers: 1 2 3 4 5 6 7 8 9 10 11 12 13 14
```

```r
# ========================================
# 2. LAB DATA
# ========================================
# Single Target Lab (Using the matrix columns)
single_lab_count <- ncol(single_levelwise_rt)
cat("--- SINGLE LAB ---\n")
```

```
## --- SINGLE LAB ---
```

```r
cat("Total Number of Levels:", single_lab_count, "\n")
```

```
## Total Number of Levels: 17
```

```r
# If PsychoPy named the columns, this will show them:
cat("Level Names:", colnames(single_levelwise_rt), "\n\n")
```

```
## Level Names:
```

```r
# Multiple Target Lab (Using the matrix columns)
multi_lab_count <- ncol(multi_levelwise_rt)
cat("--- MULTIPLE LAB ---\n")
```

```
## --- MULTIPLE LAB ---
```

```r
cat("Total Number of Levels:", multi_lab_count, "\n")
```

```
## Total Number of Levels: 15
```

```r
cat("Level Names:", colnames(multi_levelwise_rt), "\n\n")
```

```
## Level Names:
```

```r
library(dplyr)

# 1. Clean the data: Keep only completed levels up to Level 10
data_game_inter <- multiple_phone %>%
  filter(tolower(as.character(Completed)) == "true") %>%
  filter(as.numeric(as.character(Level)) <= 10) %>%
  group_by(Player, Level) %>%
  summarise(
    AvgInterTargetTime.ms. = mean(AvgInterTargetTime.ms., na.rm = TRUE)
  ) %>%
  ungroup()
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Player and Level.
## i Output is grouped by Player.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Player, Level))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```
# 2. Find ONLY the "survivor" players who completed all 10 levels
complete_players_inter <- data_game_inter %>%
  group_by(Player) %>%
  tally() %>%
  filter(n == 10) %>%
  pull(Player)

# 3. Filter the dataset to keep only those balanced players and drop ghosts
balanced_inter <- data_game_inter %>%
  filter(Player %in% complete_players_inter) %>%
  mutate(
    Player = droplevels(as.factor(Player)),
    Level = droplevels(as.factor(Level))
  )

# 4. Run the Friedman Test!
cat("\n--- FRIEDMAN TEST: MULTIPLE TARGET GAME (INTER-TARGET TIME) ---\n")
```

```
##
## --- FRIEDMAN TEST: MULTIPLE TARGET GAME (INTER-TARGET TIME) ---
```

```
print(friedman.test(AvgInterTargetTime.ms. ~ Level | Player, data = balanced_inter))
```

```
##
##  Friedman rank sum test
##
## data:  AvgInterTargetTime.ms. and Level and Player
## Friedman chi-squared = 73.315, df = 9, p-value = 3.396e-12
```

```
pairwise.wilcox.test(
  x = balanced_inter$AvgInterTargetTime.ms.,
  g = balanced_inter$Level,
  paired = TRUE,
  p.adjust.method = "holm"
)
```

```
## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with zeroes
```

```
## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with ties
## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with ties
## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with ties
```

```
## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with zeroes

## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with ties
## Warning in wilcox.test.default(xi, xj, paired = paired, ...): cannot compute
## exact p-value with ties

##
##  Pairwise comparisons using Wilcoxon signed rank exact test
##
## data:  balanced_inter$AvgInterTargetTime.ms. and balanced_inter$Level
##
##    1     2     3     4     5     6     7     8     9
## 2  0.504 -     -     -     -     -     -     -     -
## 3  1.000 0.260 -     -     -     -     -     -     -
## 4  0.044 0.096 0.082 -     -     -     -     -     -
## 5  0.082 0.044 0.483 0.112 -     -     -     -     -
## 6  0.150 0.044 0.614 0.044 1.000 -     -     -     -
## 7  0.096 0.044 0.061 1.000 0.260 0.044 -     -     -
## 8  0.044 0.044 0.315 0.483 1.000 1.000 0.483 -     -
## 9  0.061 0.044 0.391 0.082 1.000 1.000 0.044 1.000 -
## 10 0.044 0.044 0.061 1.000 0.205 0.044 1.000 0.205 0.044
##
## P value adjustment method: holm
```

```r
# ----------------------------------------------------------
# 1. EXTRACT INTER-TARGET TIME FROM MULTI LAB FILES
# ----------------------------------------------------------
multi_lab_inter_rt_matrix <- NULL

for (file in files) {
  df <- read.csv(file, stringsAsFactors = FALSE)
  player_inter_times <- c()

  entries <- df$mouse.time
  entries <- entries[!is.na(entries)]

  for (entry in entries) {
    # Clean the string and split into numeric vector
    clean <- gsub("\\[|\\]", "", entry)
    if (nchar(clean) > 0) {
      timestamps <- as.numeric(strsplit(clean, ",")[[1]])

      # Inter-target time only exists if there's more than 1 click
      if (length(timestamps) > 1) {
        # diff() calculates the time BETWEEN clicks
        avg_inter <- mean(diff(timestamps))
        player_inter_times <- c(player_inter_times, avg_inter)
      } else {
        player_inter_times <- c(player_inter_times, NA)
      }
    }
```

```
  }
  # Add this player's row to our matrix
  multi_lab_inter_rt_matrix <- rbind(multi_lab_inter_rt_matrix, player_inter_times)
}

# ------------------------------------------------------------
# 2. FORMAT AND RUN FRIEDMAN TEST
# ------------------------------------------------------------
library(tidyr)
library(dplyr)

# Convert to dataframe and add Player IDs
lab_inter_df <- as.data.frame(multi_lab_inter_rt_matrix)
lab_inter_df$Player <- as.factor(1:nrow(lab_inter_df))

# Pivot and find balanced levels (levels where all players have data)
lab_inter_long <- lab_inter_df %>%
  pivot_longer(cols = -Player, names_to = "Level_Raw", values_to = "InterRT") %>%
  mutate(Level = as.numeric(gsub("V", "", Level_Raw))) %>%
  drop_na(InterRT)

# Find common levels completed by all 21 players
total_p <- length(unique(lab_inter_long$Player))
complete_levels <- lab_inter_long %>%
  group_by(Level) %>%
  tally() %>%
  filter(n == total_p) %>%
  pull(Level)

# Filter for balanced data
balanced_lab_inter <- lab_inter_long %>%
  filter(Level %in% complete_levels) %>%
  mutate(Player = as.factor(Player), Level = as.factor(Level))

cat("\n--- FRIEDMAN TEST: MULTIPLE LAB (INTER-TARGET TIME) ---\n")
```

```
##
## --- FRIEDMAN TEST: MULTIPLE LAB (INTER-TARGET TIME) ---
```

```
if(length(complete_levels) > 1) {
  cat("Analyzing balanced data for Levels:", paste(complete_levels, collapse=", "), "\n")
  print(friedman.test(InterRT ~ Level | Player, data = balanced_lab_inter))
} else {
  cat("Error: Not enough overlapping levels found for a balanced test.\n")
}
```

```
## Analyzing balanced data for Levels: 1, 2, 3, 4, 5, 6, 7
##
##  Friedman rank sum test
##
## data:  InterRT and Level and Player
## Friedman chi-squared = 7.9554, df = 6, p-value = 0.2414
```